INTEGRAL DRAGON

MODULE 2 • SESSION 05

Spec-Driven Development

The spec is the source of truth, not the prompt.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

Most AI misses are specification misses

- The model usually fails where intent is missing, not where syntax is hard.
- Review pain starts upstream: vague tickets create vague plans, vague diffs, and vague tests.
- This cohort already uses AI often. The next maturity jump is better requirement shaping.

Vibe coding optimizes for motion. SDD optimizes for convergence.

Vibe Coding

Prompt → inspect → retry → patch → hope. Motion is easy to produce. Correct output is not.

Spec-Driven Development

Define target behavior, constraints, and checks *before* code exists. Then execute once.

The cost of iteration without intent

Each retry loop burns review time, builds technical debt, and drifts further from the actual requirement.

The win

Not formality for its own sake. Fewer loops, more predictable output, reviewable diffs from the first attempt.

A useful engineering spec is compact and executable

Goal

What outcome must exist when the work is done.

Constraints

What the agent must not violate — architecture, schema, performance, policy.

Interfaces

APIs, files, schemas, commands, or contracts touched or produced.

Acceptance Criteria

Binary checks that let a reviewer say pass or fail without asking the author.

Non-Goals

What is explicitly out of scope. Keeps the agent from widening the blast radius.

Good acceptance criteria are observable

WEAK

"make auth more robust"

WEAK

"improve performance"

BETTER

"expired bearer token returns 401 with the existing error envelope"

BETTER

"no database schema changes; existing integration tests continue to pass"

- A criterion is good if a reviewer can check it without talking to the author.
- If it requires judgment to evaluate, it is still a goal, not a criterion.

Write what must be true before guessing how it should be built

- Lead with behavior, boundaries, and risk.
- Add implementation constraints only when they are real requirements — not preferences.

Valid constraint

"Must use existing audit logger" — this is a real invariant the agent must respect.

Design advice, not a criterion

"Should probably use a service class" — valid opinion, wrong place for it.

Minimum viable spec template

1. **Problem statement** — what is broken or missing
2. **Current boundary or affected area** — what code is in scope
3. **Inputs and outputs** — what the system receives and produces
4. **Constraints and invariants** — what must remain true
5. **Edge cases and failure modes** — known exceptions to handle
6. **Acceptance checks** — binary pass/fail criteria
7. **Non-goals** — explicit scope boundaries

Repo reconnaissance comes before finalizing the spec

- Read the current code before freezing the request.
- Ask Claude Code to map the auth flow, touched files, and existing conventions.
- Then write the spec against the system that exists, not the system you imagine.

Why it matters

Specs written without reading the codebase encode imaginary constraints and miss real ones. Reconnaissance turns guesses into facts before any generation begins.

A short SDD loop beats a giant requirements ceremony

01
Ticket or idea arrives

02
Read the boundary and affected code

03
Draft the spec

04
Critique and compress

05
Freeze the acceptance checks

06
Hand the spec to planning

→ A clear spec can usually be sliced into reviewable work packets: boundary mapping, implementation, tests.

LIVE DEMO

Turn a weak ticket into an executable spec

Prompt → critique → rewrite → binary criteria

Prompt flow

```
$ claude

# Step 1
"Critique this ticket for ambiguity and hidden assumptions."

# Step 2
"Rewrite it as a compact markdown spec with binary acceptance criteria."

# Step 3
"List open questions that still block safe planning."
```

- → Watch whether Claude surfaces missing business rules or quietly invents them.
- → Are the acceptance checks observable by a reviewer? Could this spec feed Plan Mode?

Common mistakes when writing specs

Treating a user story as a spec

User stories express intent. Specs express observable behavior and constraints. They are different artifacts.

Implementation smuggled in

Embedding design guesses in the acceptance section — now the agent is constrained by your preference, not the requirement.

Untestable criteria

Writing criteria that sound good but cannot be checked by a reviewer without interpretation.

Missing non-goals

Forgetting to scope out adjacent concerns lets the agent widen scope silently.

KEY TAKEAWAY

A prompt can start work.
A spec can govern it.

In enterprise delivery, governing work is the job.
Session 06 next: Plan Mode turns the spec into a reviewable execution proposal.